"Convention over configuration — write less, deliver more."

MODULE 23

Spring Boot Auto-Configuration

Build production-ready Spring applications faster with starters, auto-configuration, embedded servers, and Actuator.

TEKsystems Global Services

WORKFORCE DEVELOPMENT

MODULE 23 OVERVIEW

Build production-ready Spring applications faster with starters, auto-configuration, embedded servers, and Actuator.

Spring Boot replaces the manual wiring of Module 22 with convention over configuration -- this module covers starters, auto-configuration, embedded servers, and Actuator, then bootstraps Northstar CRM's first runnable Boot application.

DURATION & LAB

4 Hours Theory

Spring Boot fundamentals, auto-configuration internals, starter dependencies, and the application startup lifecycle.

2 Hours Lab

Northstar CRM First Boot App: scaffold a runnable service with REST customer endpoints and Actuator health.

Scenario

A team must launch a new service in minutes, not days -- starters and auto-configuration replace manual XML setup.

MODULE ARC

Understand Auto-Configuration

Classpath detection drives conditional configuration -- no more manual bean wiring.

Master Starters & Actuator

Starter dependencies, embedded Tomcat, and production-ready health and metrics endpoints.

Build the Lab

Scaffold Northstar CRM's first Boot app with application.yml, REST customers, and Actuator health.

KEY TAKEAWAY Total time: 4 hours theory + 2 hours hands-on lab. Spring Boot's auto-configuration does the heavy lifting so you can focus on business logic -- production-ready with minimal effort.

WHY SPRING BOOT CHANGED ENTERPRISE DEVELOPMENT

Spring Boot revolutionized enterprise Java development by making it faster, simpler, and production-ready by default.

TRADITIONAL SPRING (BEFORE)

- **Complex Configuration —** XML, multiple setup files, and boilerplate code slowed development.
- **Dependency Management —** manual management led to version conflicts and inconsistent environments.
- **External Servers —** required external application servers (Tomcat, JBoss) for deployment.
- **More Infrastructure Effort —** developers spent more time on setup, wiring, and environment management.
- **Slower Time to Market —** more effort, more complexity, longer release cycles.

SPRING BOOT (AFTER)

- **Auto-Configuration —** smart defaults eliminate manual setup and boilerplate code.
- **Starter Dependencies —** curated starters simplify build configuration and ensure compatibility.
- **Embedded Servers —** built-in servers (Tomcat, Jetty, Undertow) enable deploy-anywhere apps.
- **Developer Productivity —** convention over configuration lets you focus on business logic.
- **Faster Time to Market —** rapid development, easy testing, streamlined deployment.

ENTERPRISE IMPACT

Higher Productivity | **Reliability & Consistency** | **Scalability** | **Lower Operational Cost**

KEY TAKEAWAY Spring Boot brought simplicity, speed, and stability to enterprise Java — less configuration, more innovation.

MODULE LEARNING OBJECTIVES

By the end of this module, you will be able to:

- **Understand Spring Boot Fundamentals —** explain what Spring Boot is, how it differs from Spring, and its key benefits.
- **Explain Auto-Configuration —** understand the concept, how it works, and the role of conditional configuration.
- **Work with Starter Dependencies —** identify and use starters to simplify and accelerate development.
- **Create Projects with Spring Initializr —** generate and understand a Spring Boot project's structure and generated files.
- **Understand the Application Lifecycle —** describe SpringApplication, embedded servers, and the startup sequence.
- **Use Actuator for Monitoring —** enable and explore health, metrics, and other production endpoints.
- **Apply Best Practices —** follow production-readiness guidelines for enterprise applications.

KEY TAKEAWAY These objectives build the skills to create, configure, run, and monitor production-ready Spring Boot applications.

WHAT IS SPRING BOOT?

An opinionated Java framework built on Spring that makes it easy to create stand-alone, production-ready applications.

Spring Boot is an open-source Java framework built on top of the Spring Framework that makes it easy to create stand-alone, production-ready, Spring-based applications with minimal configuration and setup.

KEY POINTS

Convention over Configuration

Provides sensible defaults and reduces manual configuration.

Opinionated

Follows best practices and a structured way to build applications.

Standalone

Applications run with an embedded server (Tomcat, Jetty, Undertow).

Production Ready

Built-in Actuator, health checks, metrics, externalized config.

WHY IT EXISTS

- To speed up development and reduce boilerplate configuration.
- To simplify dependency management with starter dependencies.
- To run applications as standalone services without external server setup.
- To make applications production-ready out of the box.

KEY TAKEAWAY Spring Boot helps you build and run Spring applications faster with less configuration.

SPRING VS SPRING BOOT

Spring Boot is built on top of the Spring Framework to simplify development, reduce configuration, and accelerate productivity.

Aspect	Spring (Traditional)	Spring Boot
Configuration	Requires extensive XML/Java configuration and manual setup.	Auto-configuration eliminates most XML and manual setup.
Dependency Mgmt	Manual dependency management; higher chance of version conflicts.	Starter dependencies simplify setup with compatible versions.
Server	Requires external application servers (Tomcat, JBoss, etc.).	Comes with embedded servers (Tomcat, Jetty, Undertow) to run anywhere.
Development Speed	More boilerplate code and configuration to get started.	Less code, less configuration — start coding business logic faster.
Built-in Features	Manual setup for logging, security, metrics, etc.	Provides production-ready features like Actuator, health checks, metrics.
Deployment	Harder to deploy and scale consistently across environments.	Easy to package (JAR), deploy, and scale in any environment.
Time to Market	More setup, more time before a working application.	Rapid development and quick time to market.

KEY TAKEAWAY Spring Boot uses the power of Spring and makes it easy, fast, and production-ready.

Spring vs Spring Boot

Spring Boot builds on top of the Spring Framework to simplify development and accelerate delivery.

SPRING FRAMEWORK		SPRING BOOT
CONFIGURATION Requires extensive XML or Java configuration and manual setup.	Configuration	**AUTO-CONFIGURATION** Automatic configuration based on classpath and sensible defaults.
DEPENDENCY MANAGEMENT Manual addition and management of dependencies.	Dependencies	**STARTER DEPENDENCIES** Provides starter dependencies to simplify dependency management.
SERVER Requires external server (e.g., Tomcat) for deployment.	Server	**EMBEDDED SERVER** Comes with embedded servers like Tomcat, Jetty, or Undertow.
DEVELOPMENT More boilerplate code and repetitive configuration.	Productivity	**DEVELOPER PRODUCTIVITY** Reduces boilerplate, speeds up development with conventions over configuration.
DEPLOYMENT Requires packaging as WAR and deploying to external server.	Deployment	**STANDALONE APPLICATION** Package as JAR and run as a standalone application.

SPRING FRAMEWORK
Powerful and flexible framework for building enterprise applications.

BOTTOM LINE
Spring Boot = Convention + Auto-Configuration + Embedded = Faster Delivery

SPRING BOOT
Opinionated, production-ready framework that simplifies and accelerates development.

BENEFITS OF SPRING BOOT

Spring Boot brings convention over configuration and a production-ready approach — faster, less effort, greater confidence.

Faster Development

Auto-configuration, starters, and embedded servers cut setup time.

Reduced Configuration

Eliminates extensive XML; sensible defaults minimize manual setup.

Simplified Dependencies

Starters manage compatible versions and transitive dependencies.

Standalone Apps

Built-in embedded servers let apps run anywhere.

Production Ready

Built-in Actuator, health checks, metrics, external config.

Higher Productivity

Focus more on business logic, less on infrastructure.

Security by Default

Secure defaults and easy integration with Spring Security.

Easy Monitoring

Actuator endpoints, metrics, and logging integrations.

Cloud & Containers

Designed for cloud-native environments and containers.

Microservices Ready

Lightweight, modular, easy to break into microservices.

KEY TAKEAWAY Spring Boot accelerates delivery of reliable, scalable, and maintainable applications with minimal effort.

SPRING BOOT ARCHITECTURE

Spring Boot is built on top of the Spring Framework and provides auto-configuration, embedded servers, and production-ready features.

Layer	Key Components	Responsibility
Client Layer	Web Browser, Mobile App, External Clients	Initiates requests to the application.
Web Layer	Spring MVC / REST Controllers	Handles HTTP request/response routing.
Application Layer	Service Layer, Repository Layer, Domain/Model	Business logic, data access, entities and DTOs.
Boot Infrastructure	Auto-Configuration, Starters, Embedded Server, Actuator	Auto-configures beans, starts the server, exposes monitoring.
Core Container	Spring Framework Core — IoC, AOP, Transactions, Data, Security	The foundation every other layer builds on.

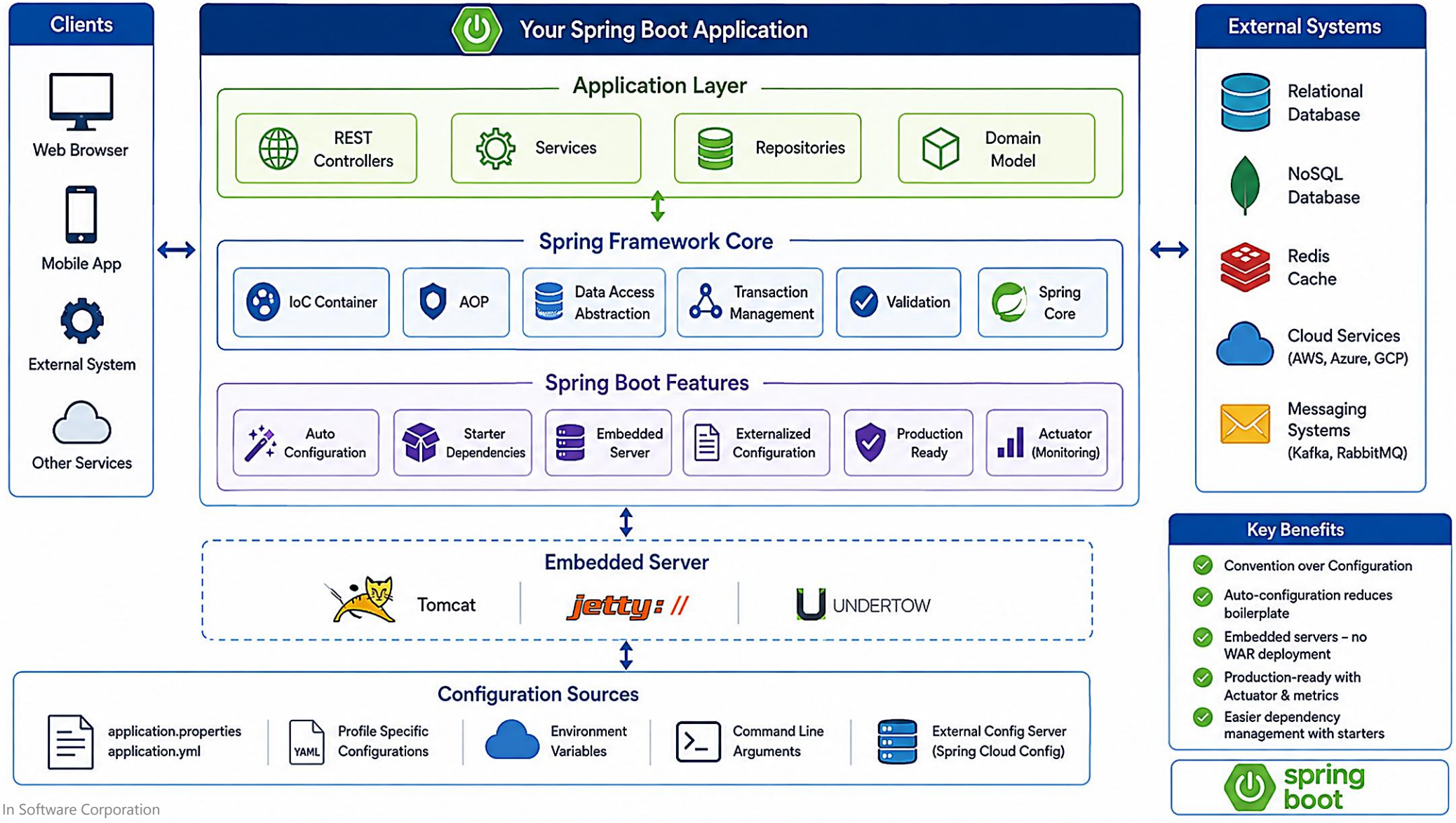
HOW IT WORKS

1. SpringApplication starts → **2. Auto-config scans classpath** → **3. Embedded server starts** → **4. Application ready for requests**

KEY TAKEAWAY Spring Boot layers auto-configuration, starters, embedded servers, and Actuator on top of the Spring Framework core.

Spring Boot Architecture

Spring Boot builds on top of the Spring Framework and provides everything you need to create, run, and monitor production-ready applications with minimal configuration.

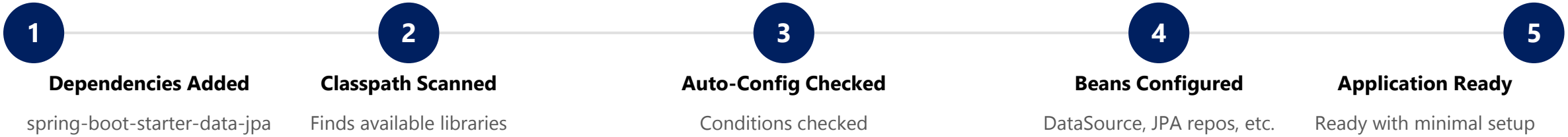


UNDERSTANDING AUTO-CONFIGURATION

Auto-Configuration is the heart of Spring Boot — it configures your application based on the libraries you add and the settings you provide.

WHAT DOES IT DO?

- **Configures beans automatically —** creates and configures Spring beans for you.
- **Configures infrastructure —** data sources, JPA, web server, security, Actuator, etc.
- **Applies sensible defaults —** uses best practices to provide default properties and behavior.
- **Conditional and smart —** only configures what is needed and backs off if you provide your own configuration.



KEY TAKEAWAY Auto-Configuration = Convention over Configuration — Spring Boot figures out what you need and configures it automatically.

Spring Boot Auto-Configuration

Spring Boot automatically configures your application based on the dependencies on the classpath, the beans you define, and the property settings you provide.

WHAT IS AUTO-CONFIGURATION?

Automatically configures Spring application context based on:

- Classpath dependencies
- Existing beans
- Configuration properties

KEY BENEFITS

- Reduces boilerplate configuration
- Speeds up development
- Follows best practices by default
- Easier to build production-ready applications

HOW AUTO-CONFIGURATION WORKS

1. **APPLICATION STARTUP** — SpringApplication starts and loads the application context.
2. **AUTO-CONFIGURATION CANDIDATES LOADED** — Spring Boot loads all auto-configuration classes listed in META-INF/spring/org.springframework.boot.autoconfigure.AutoConfiguration.imports
3. **CONDITIONS EVALUATED** — Each auto-configuration class is evaluated against conditions (@Conditional... annotations).
4. **BEANS CONFIGURED** — If conditions match, Spring creates and configures the required beans.
5. **APPLICATION READY** — The application context is fully initialized and ready to use.

CONDITIONAL CONFIGURATION

Auto-configuration classes use conditions to decide whether to apply.

@ConditionalOnClass	Applies if a specific class is present on the classpath.
@ConditionalOnMissingBean	Applies if a specific bean is NOT already defined.
@ConditionalOnProperty	Applies based on a property value in application.properties.
@ConditionalOnWebApplication	Applies if the application is a web application.
@ConditionalOnResource	Applies if a specific resource is available.

EXAMPLE: WEB STARTER AUTO-CONFIGURATION

DEPENDENCIES ON CLASSPATH
Spring-boot-starter-web (brings Tomcat, Spring MVC, etc.)

CONDITIONS CHECKED
- Is DispatcherServlet class present?
- Is no existing DispatcherServlet bean defined?
- Is this a web application?
- Are required properties available?

AUTO-CONFIGURATION APPLIED
DispatcherServlet, ViewResolver, MessageConverters, and other beans are auto-configured.

RESULT
A fully configured web application ready to run.

AUTO-CONFIGURATION IN ACTION

DATA ACCESS
Configures DataSource, EntityManager, Transaction Manager, etc.

SECURITY
Configures Spring Security filter chain and default security rules.

ACTUATOR
Exposes health, metrics, info endpoints automatically.

MAIL
Configures JavaMailSender when mail properties are provided.

BOTTOM LINE

Auto-configuration removes the need for manual configuration.
You focus on business logic, Spring Boot handles the rest!

spring boot

HOW AUTO-CONFIGURATION WORKS

Spring Boot auto-configures your application by detecting what is on the classpath, checking conditions, and configuring beans.

EXAMPLE: spring-boot-starter-web

You add spring-boot-starter-web -> Spring Boot finds Tomcat, Spring MVC, and Jackson on the classpath -> web-application conditions match -> it auto-configures Tomcat, DispatcherServlet, and MessageConverters -> the application starts with an embedded Tomcat and sensible defaults.

IF YOU DEFINE YOUR OWN BEAN...

Spring Boot backs off and does not create its own auto-configured bean — your configuration always takes priority.

KEY TAKEAWAY Spring Boot looks at what you have, figures out what you need, checks conditions, and configures everything automatically.

How Spring Boot Auto-Configuration Works

Spring Boot intelligently configures your application based on what's on the classpath, the beans you define, and the properties you set.

1 Application Starts

SpringApplication.run() starts the application and initializes the ApplicationContext.

2 Auto-Configuration Classes Loaded

Spring Boot loads all auto-configuration classes listed in META-INF/spring/org.springframework.boot.autoconfigure.AutoConfiguration.imports

3 Conditions Evaluated

Each auto-configuration class has conditions (@Conditional...) that are evaluated against:

- Classpath
- Existing beans
- Properties
- Environment

4 Matching Conditions Applied

If all conditions match, Spring Boot applies the configuration and registers the required beans in the context.

5 Application Context Ready

The ApplicationContext is fully initialized with the auto-configured beans. Your application is ready to use.

Conditional Decision Examples

@ConditionalOnClass
Applies if a specific class is present on the classpath. (e.g., DataSource class)

@ConditionalOnBean
Applies if a specific bean is already defined in the context.

@ConditionalOnProperty
Applies if a property with a specific value exists. (e.g., server.port)

@ConditionalOnWebApplication
Applies if the application is a web application.

@ConditionalOnMissingBean
Applies if a specific bean is NOT already defined.

EXAMPLE: DataSource Auto-Configuration

Dependency on Classpath
spring-boot-starter-jdbc added to classpath

Conditions Checked
- DataSource class present
- No existing DataSource bean
- Required properties available

Auto-Configuration Applied
Spring Boot creates and configures DataSource, JdbcTemplate, TransactionManager, etc.

Result
Ready to use with zero manual configuration

Key Benefits

- Zero boilerplate configuration
- Faster development
- Consistent best practices
- Smart defaults with the ability to customize

Behind the Scenes

1. Auto-configuration classes are discovered at runtime.
2. Each class declares @Conditional annotations.
3. Spring Boot evaluates conditions using ConditionEvaluator.
4. Matching configurations contribute bean definitions.
5. Bean definitions are registered in the ApplicationContext.

spring boot

BOTTOM LINE

Spring Boot auto-configuration removes manual setup by automatically configuring your application based on intelligent conditions, so you can focus on building features, not infrastructure.

Spring Boot Auto-Configuration Workflow

Spring Boot automatically configures your application based on the classpath, configuration and declared beans.

1 Application Start

SpringApplication.run() is called.

Bootstrap environment and listeners are initialized.

2 Load Auto-Configuration Candidates

Spring Boot loads potential auto-configuration classes from META-INF/spring/org.springframework.boot.autoconfigure.AutoConfiguration imports file.

3 Evaluate Conditions

Each auto-configuration class is evaluated using @Conditional* annotations (e.g., @ConditionalOnClass, @ConditionalOnProperty, @ConditionalOnMissingBean, @ConditionalOnWebApplication, etc.).

4 Apply Auto-Configuration

Configurations that match the conditions are applied.

Feature beans are registered in the ApplicationContext.

5 User Beans Take Priority

If user-defined beans exist, they override auto-configured beans.

(@ConditionalOnMissingBean ensures this behavior.)

6 Application Context Ready

ApplicationContext is fully initialized with auto-configured and user-defined beans.

Application is ready to use.

Result → A fully configured and ready-to-run application with minimal manual configuration.

HOW AUTO-CONFIGURATION WORKS

Classpath Detection
Spring Boot detects libraries on the classpath.

Conditional Evaluation
Conditions are evaluated to decide what to configure.

Configuration
Matching configurations are applied automatically.

Bean Registration
Beans are created and registered in the context.

KEY SOURCES

Auto-Configuration classes
`spring-boot-autoconfigure module`

Import file
`META-INF/spring/org.springframewerk.boot.autoconfigure.AutoConfiguration`

Conditional Annotations
@ConditionalOnClass, @ConditionalOnProperty, @ConditionalOnMissingBean, etc.

EXAMPLES OF AUTO-CONFIGURED FEATURES

DataSource · JPA / Hibernate · Spring Security · Mail Sender · Actuator / Monitoring · Jackson / JSON · Web Server (Tomcat/Jetty/Undertow) · And many more...

BENEFITS

- Saves development time
- Reduces boilerplate configuration
- Follows best practices by default
- Easy to extend and customize
- Focus on business logic, not infrastructure

spring boot

CONDITIONAL CONFIGURATION

Spring Boot uses conditions to decide whether a particular auto-configuration class should be applied or skipped.

WHY IT MATTERS

- Ensures only the right beans are created — no conflicts or unnecessary configuration.
- Enables smart, context-aware auto-configuration based on what is actually present.

Annotation	What It Checks
@ConditionalOnClass	Checks if a specific class is present on the classpath.
@ConditionalOnMissingClass	Checks if a specific class is NOT present.
@ConditionalOnBean	Checks if a specific bean exists in the context.
@ConditionalOnMissingBean	Checks if a specific bean does NOT exist.
@ConditionalOnProperty	Checks if a property has a specific value.
@ConditionalOnWebApplication	Checks if the application is a web application.

Example: @ConditionalOnClass

```
@Configuration
@ConditionalOnClass(name = "org.springframework.data.jpa.repository.JpaRepository")
public class JpaRepositoriesAutoConfiguration { /* configures JPA repositories */ }
```

KEY TAKEAWAY Conditional configuration is the intelligence behind auto-configuration — exactly what you need, nothing more.

Spring Boot Conditional Configuration

Spring Boot auto-configuration classes are applied only when specific conditions are met.

WHAT IS CONDITIONAL CONFIGURATION?

Spring Boot uses conditions to decide whether to apply a particular auto-configuration.

If the conditions match, the configuration is applied. If not, it is skipped.

HOW CONDITIONAL CONFIGURATION WORKS

1 Auto-Configuration Class Loaded

Spring Boot loads auto-configuration classes from META-INF/spring/org.springframework.boot.autoconfigure.AutoConfiguration.imports

2 Conditions Declared

The class declares one or more @Conditional annotations that specify when it should be applied.

3 Condition Evaluation

Spring Boot evaluates all conditions against the current application context, environment, and classpath.

4 Match or No Match

If all conditions match → configuration is applied.

If any condition fails → configuration is skipped.

5 Application Context

Matching configurations contribute beans to the ApplicationContext. Skipped ones are ignored.

WHY CONDITIONAL?

- Avoids unnecessary beans
- Reduces conflicts
- Provides sensible defaults
- Makes auto-configuration smart and flexible
- You can still override when needed

COMMON CONDITIONAL ANNOTATIONS

		Example
@ConditionalOnClass	Matches if the specified class(es) are present on the classpath.	@ConditionalOnClass(DataSource.class)
@ConditionalOnBean	Matches if a specific bean is already defined in the context.	@ConditionalOnBean(DataSource.class)
@ConditionalOnProperty	Matches if a property has a specific value.	@ConditionalOnProperty(prefix="app", name="feature", havingValue="true")
@ConditionalOnWebApplication	Matches if the application is a web application.	@ConditionalOnWebApplication(type = Type.SERVLET)
@ConditionalOnMissingBean	Matches if no bean of the specified type is present.	@ConditionalOnMissingBean(MyService.class)
@ConditionalOnCloudPlatform	Matches if the application is running on a cloud platform.	@ConditionalOnCloudPlatform(CloudPlatform.AWS)

EXAMPLE: DataSourceAutoConfiguration

CONDITIONS DECLARED

- @ConditionalOnClass(DataSource.class)
- @ConditionalOnMissingBean(type = DataSource.class)
- @ConditionalOnProperty(prefix = "spring.datassource", name = "url")

ALL CONDITIONS MATCH

DataSourceAutoConfiguration is applied.

DataSource bean is created with properties.

ANY CONDITION FAILS

DataSourceAutoConfiguration is skipped.

No DataSource bean is created.

spring boot

BOTTOM LINE

Conditional configuration ensures that Spring Boot applies the right configuration at the right time based on your environment, dependencies, and settings.

KEY TAKEAWAY

Conditions make auto-configuration intelligent, modular, and non-intrusive.

TRY IT YOURSELF — EXERCISE 1: AUTO-CONFIG VERSUS OWNERSHIP

Reinforces: auto-configuration gifts vs. what you still own — the conditional configuration you just covered.

STEP	WHAT TO DO
Goal	Produce a three-and-three list Lab 23 expects: Boot's auto-config gifts vs. what Northstar still designs.
File	notes/autoconfig-ownership.md (in examples/module-23-exercises/)
Auto-config gifts	List three Boot gifts, e.g. embedded Tomcat, DispatcherServlet wiring, Jackson JSON converters.
You still own	List three items Northstar designs: API paths/status codes, domain validation rules, DTO field exposure policy.
CRM fixtures	Mention smoke targets: POST/GET CUS-1001, CUS-1002, correlation lab-request-001.
Reference	exercises/exercise-01-autoconfig-vs-ownership.md

Reference: Gift vs. Ownership (notes/autoconfig-ownership.md)

```
$ cat notes/autoconfig-ownership.md
Embedded Tomcat          -> API paths and status codes
DispatcherServlet wiring -> Domain validation rules
Jackson JSON converters  -> DTO field exposure policy
// smoke targets: POST/GET CUS-1001, CUS-1002 (correlation lab-request-001)
```

TRY IT NOW Complete Exercise 1 before continuing — see exercises/exercise-01-autoconfig-vs-ownership.md.

STARTER DEPENDENCIES — HOW STARTERS WORK

Starters are convenient dependency descriptors that bring in all the libraries you need for a specific feature.

Instead of adding many individual dependencies (spring-web, jackson-databind, tomcat-embed-core, validation-api...), you add one starter.

KEY IDEA

- **One starter = many related libraries —** reduces complexity and ensures compatible versions.
- **Spring Boot automatically adds —** Spring libraries, third-party libraries, logging, JSON support, validation, and an embedded server.

WHAT IT LETS YOU FOCUS ON

Business Logic | **Not Boilerplate** | **Not Version Conflicts** | **Not Manual Wiring**

NAMING CONVENTION

spring-boot-starter-xxx — always starts with "spring-boot-starter"; xxx names the feature or technology it provides (e.g. web, data-jpa, security).

KEY TAKEAWAY Starters are shortcut packages that bundle everything you need for a feature so you can start building quickly.

STARTER DEPENDENCIES — POPULAR STARTERS

A curated catalog of starters covers most common enterprise application needs.

Starter	What It Provides
spring-boot-starter-web	Build web applications, REST APIs, and MVC apps.
spring-boot-starter-data-jpa	JPA with Hibernate and Spring Data.
spring-boot-starter-data-mongodb	MongoDB support.
spring-boot-starter-security	Spring Security.
spring-boot-starter-validation	Bean validation (JSR-380).
spring-boot-starter-test	Testing support (JUnit, Mockito, AssertJ).

Add to pom.xml

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>
    spring-boot-starter-web
  </artifactId>
</dependency>
```

BENEFITS OF STARTERS

- Simplifies dependency management.
- Ensures compatible library versions.
- Reduces configuration and boilerplate.
- Boosts productivity and best practices.

KEY TAKEAWAY Use starters whenever possible — they save time, reduce errors, and ensure compatible dependency versions.

Spring Boot Starter Dependencies

Starters are a set of convenient dependency descriptors that you can include in your application.
They simplify your build configuration and save development time.

WHAT IS A STARTER?

A starter is a curated set of dependencies that work well together for a specific purpose.

It eliminates the need to add multiple individual dependencies manually.

HOW STARTERS WORK

1. **Add Starter Dependency** — Add a starter dependency in your build file (Maven/Gradle).
2. **Transitive Dependencies Included** — Spring Boot brings in all required dependencies transitively.
3. **Auto-Configuration Applied** — Auto-configuration kicks in and configures the beans automatically.
4. **Minimal Configuration** — You focus on business logic, not on complex setup.
5. **Application Ready** — Your application is up and running quickly and consistently.

BENEFITS

- Speeds up development
- Reduces dependency management
- Promotes best practices
- Consistent and production-ready by default
- Easier upgrades and maintenance

POPULAR SPRING BOOT STARTERS

spring-boot-starter-web
Build web, including RESTful, applications using Spring MVC.
Includes:
- spring-web
- spring-webmvc
- jackson-databind
- embedded Tomcat

spring-boot-starter-data-jpa
Persist data using JPA with Spring Data.
Includes:
- spring-data-jpa
- hibernate-core
- spring-orm

spring-boot-starter-data-jdbc
JDBC support with connection pool and data access.
Includes:
- spring-jdbc
- HikariCP

spring-boot-starter-security
Add Spring Security authentication and authorization.
Includes:
- spring-security-core
- spring-security-config
- spring-security-web

spring-boot-starter-test
Testing support with JUnit, Mockito, and more.
Includes:
- junit-jupiter
- mockito-core
- assertj-core
- spring-test

spring-boot-starter-validation
Bean validation with Hibernate Validator.
Includes:
- spring-boot-starter
- hibernate-validator

spring-boot-starter-actuator
Production-ready features to monitor and manage your application.
Includes:
- spring-boot-actuator
- micrometer-core

spring-boot-starter-mail
Send emails using Spring's mail support.
Includes:
- spring-context-support
- spring-mail

spring-boot-starter-thymeleaf
Server-side rendering with Thymeleaf.
Includes:
- thymeleaf
- thymeleaf-spring

spring-boot-starter-amqp
Messaging support with RabbitMQ.
Includes:
- spring-rabbit
- spring-amqp

KEY TAKEAWAY

Starters bundle the right dependencies and configurations so you can build applications faster and with confidence.

Use starters, write less configuration, and build production-ready applications!

EXAMPLE (MAVEN)

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-web</artifactId>
</dependency>
```

TRY IT YOURSELF — EXERCISE 2: BOOT STARTERS INVENTORY

Reinforces: the starter catalog and naming convention you just covered.

STEP	WHAT TO DO
Goal	Map Boot starters (web, actuator, test) to Northstar CRM capabilities for Lab 23.
File	notes/starters.md (in examples/module-23-exercises/)
Starter meanings	One sentence each for web, actuator, and test — fix any 'starter = whole framework' overclaims.
Ownership boundary	List three things Boot does NOT own: customer lifecycle rules, validation messages, which endpoints are public.
Pre-lab boundary	Note you will not run spring-boot:run for the full lab here — this is a reading/writing exercise.
Reference	exercises/exercise-02-starters-inventory.md

Reference Table (notes/starters.md)

```
$ cat notes/starters.md
spring-boot-starter-web       -> Embedded server + Spring MVC
spring-boot-starter-actuator  -> Health and ops endpoints
spring-boot-starter-test      -> JUnit / MockMvc / context tests
// Boot does NOT own: customer lifecycle rules, validation messages, public endpoints
```

TRY IT NOW Complete Exercise 2 before continuing — see exercises/exercise-02-starters-inventory.md.

CREATING PROJECTS WITH SPRING INITIALIZR

A web-based tool that quickly bootstraps a Spring Boot project with the right dependencies and configuration.

1. **Open Initializr** — start.spring.io
2. **Configure Metadata** — Group, artifact, name
3. **Select Dependencies** — Search what you need
4. **Packaging & Java** — Jar/War, Java version
5. **Generate & Download** — ZIP project

EXAMPLE CONFIGURATION

- Project: Maven · Language: Java · Java: 21
- Spring Boot: 3.2.5 (default) · Packaging: Jar
- Group: com.example · Artifact: demo-app
- Dependencies: Spring Web, Spring Data JPA, DevTools

WHAT YOU GET

- Complete project structure, ready to import.
- Pre-configured pom.xml / build.gradle.
- Application class with main() method.
- application.properties, ready to run and build.

KEY TAKEAWAY Spring Initializr helps you create a Spring Boot project in minutes with the right setup and dependencies.

PROJECT STRUCTURE OVERVIEW

Spring Boot projects follow a standard, convention-based structure that keeps code organized and production-ready.

Typical Directory Layout

```
my-spring-boot-app/
|-- src/main/java/com/example/myapp/
|   |-- MyAppApplication.java
|   |-- controller/
|   |-- service/
|   |-- repository/
|   |-- entity/
|   |-- dto/
|   `-- config/
|-- src/main/resources/
|   |-- application.properties
|   |-- static/
|   `-- templates/
|-- src/test/java/...
|-- pom.xml
`-- README.md
```

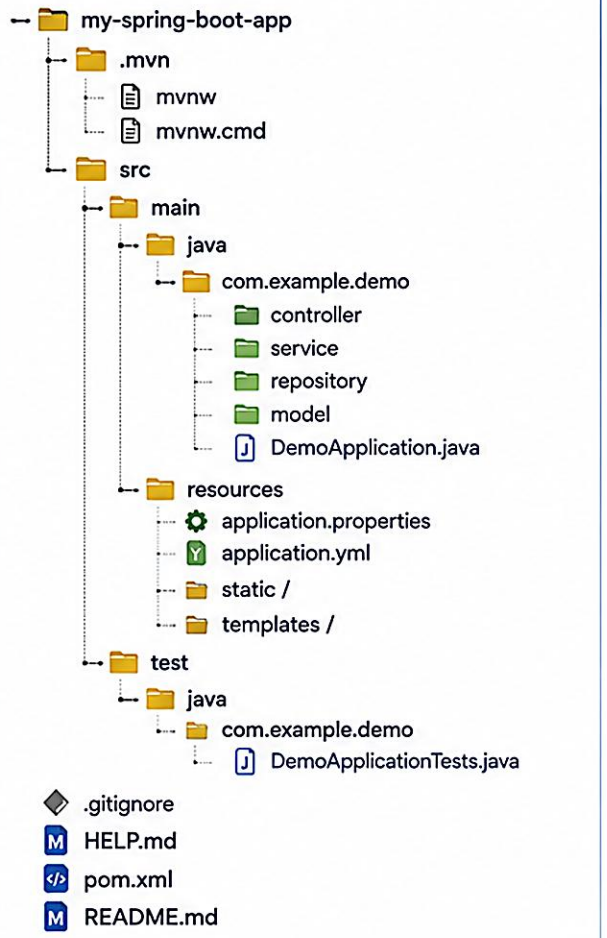
Path / File	Purpose
src/main/java	All Java source code — where application logic lives.
controller/	REST controllers that handle HTTP requests.
service/	Business logic layer.
repository/	Data access layer (Spring Data JPA repositories).
entity/ , dto/	JPA entities and Data Transfer Objects.
src/main/resources	Properties, YAML, static content, templates.
src/test/java	Test source code (unit, integration tests).
pom.xml	Build file and dependency management.

KEY TAKEAWAY The project structure is designed to separate concerns clearly — follow it, and Spring Boot does the rest.

Spring Boot Project Structure

Spring Boot follows a convention-over-configuration approach
with a standard project structure to keep your application organized and easy to maintain.

TYPICAL PROJECT STRUCTURE



WHAT EACH PART DOES

.mvn	Maven Wrapper files to run the project without installing Maven.
src/main/java	Contains the source code of your application.
com.example.demo (root package)	Base package. Keep all your application packages under this to enable component scanning.
controller	REST controllers to handle HTTP requests.
service	Business logic and service layer components.
repository	Data access layer (Spring Data JPA repositiories, etc.).
model	Entity classes and domain models.
DemoApplication.java	Main class with @SpringBootApplication annotation. Application entry point.
src/main/resources	Resource files and application configurations.
application.properties / application.yml	Application configuration files.
static/	Static resources (CSS, JS, images).
templates/	View templates (Thymeleaf, Freemarker, etc.).
src/test/java	Test source code.
*Tests.java	Unit and integration tests.
pom.xml	Maven project file with dependencies and build configuration.

KEY BENEFITS

Convention Over Configuration
Follows a standard structure so Spring Boot can auto-configure components.

Easy Navigation
Logical separation of concerns (controller, service, repository, model).

Scalability
Easy to grow and maintain as the application evolves.

Testability
Dedicated test structure for reliable and maintainable tests.

IMPORTANT GENERATED FILES

HELP.md
Auto-generated documentation for your project.

mvnw / mvnw.cmd
Maven Wrapper scripts.

.gitignore
Pre-configured to ignore unnecessary files.

README.md
Basic project overview (you can customize).

BOTTOM LINE

Spring Boot provides a clean, standard project structure so you can focus on building features, not on configuration.

TIP
Keep your main application class in the root package so component scanning works out of the box.

UNDERSTANDING GENERATED FILES

Spring Initializr generates a ready-to-use project with predefined files so you can focus on writing business logic.

File / Directory	Type	Purpose
DemoApplication.java	Java Class	Main class with the Spring Boot entry point (@SpringBootApplication, main()).
application.properties	Configuration	Primary configuration file — server, database, logging, and more.
static/	Resources	Static resources (CSS, JavaScript, images, fonts) served as-is.
templates/	Resources	View templates (Thymeleaf by default).
DemoApplicationTests.java	Test Class	Basic test class with a context-load test to verify the app starts.
pom.xml	Build File	Maven Project Object Model — dependencies, plugins, build config.
.mvn/	Maven Wrapper	Lets the project build with a pinned Maven version, no install needed.
.gitignore	Config	Specifies intentionally untracked files that Git should ignore.

KEY TAKEAWAY These generated files provide the essential scaffolding — understand their purpose, keep what you need, customize as your app evolves.

SPRINGAPPLICATION CLASS — BASIC USAGE

The heart of every Spring Boot app — it bootstraps and launches the application, starting the embedded server and ApplicationContext.

DemoApplication.java

```
package com.example.demo;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication
public class DemoApplication {
    public static void main(String[] args) {
        SpringApplication.run(DemoApplication.class, args);
    }
}
```

KEY PIECES

- **@SpringBootApplication —** enables component scanning, auto-configuration, and property support.
- **SpringApplication.run(...) —** bootstraps the app, creates/refreshes the ApplicationContext, starts the embedded server.
- **Program Entry Point —** main() delegates the entire startup process to SpringApplication.

BEST PRACTICES

- Keep the main class in the root package for proper component scanning.
- Avoid customizing SpringApplication unless you have a specific need.

KEY TAKEAWAY SpringApplication reduces complex Spring setup to a single line of code and handles everything needed to run.

TRY IT YOURSELF — EXERCISE 3: CRMAPPLICATION STUB (STARTER)

Reinforces: @SpringBootApplication and SpringApplication.run() you just covered — a fill-in-the-blank starter, not from scratch.

STEP	WHAT TO DO
Goal	Fill blanks on a @SpringBootApplication stub and a fake Actuator health line.
Starter	TODO / fill-in-the-blank skeleton — replace every _____; blanks do not compile as-is.
File	notes/CrmApplicationStub.java + notes/health-sketch.md (notes only — Lab 23 uses the real starter).
Blanks to fill	@_____ on the class; SpringApplication._____(CrmApplication.class, args) inside main().
Answer key	@SpringBootApplication; run; path health; status UP — self-check. Do not invent JWT/SOAP stubs.
Reference	exercises/exercise-03-crm-application-stub.md

notes/CrmApplicationStub.java (fill in the blanks)

```
package com.northstar.crm;
// TODO: Spring Boot annotation that enables auto-config + component scan
@_____
public class CrmApplication {
    public static void main(String[] args) {
        SpringApplication._____(CrmApplication.class, args);
    }
}
```

TRY IT NOW Complete Exercise 3 (fill in every blank, then self-check) before continuing — see exercises/exercise-03-crm-application-stub.md.

SPRINGAPPLICATION CLASS — INTERNALS & KEY METHODS

What SpringApplication.run() does internally, step by step, and the methods you can use to customize it.

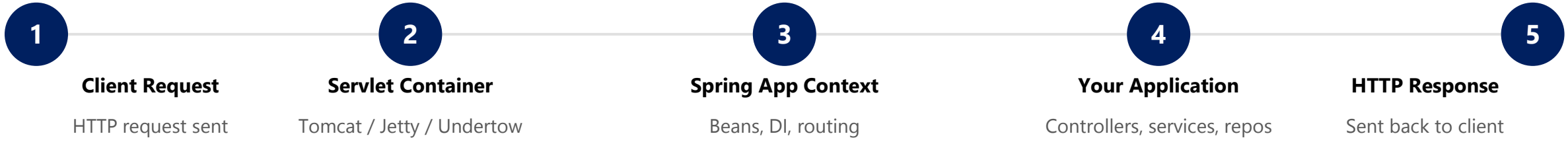
1. **Create Instance** — Configures default settings
2. **Prepare Environment** — Loads properties, YAML, args
3. **Configure Context** — Creates ApplicationContext
4. **Refresh Context** — Initializes beans, auto-config
5. **Start the Server** — Tomcat/Jetty/Undertow

Key Method (via SpringApplication instance)	Purpose
setSources(...)	Specify the primary sources to load.
setWebApplicationType(...)	Set web application type (SERVLET, REACTIVE, NONE).
setAdditionalProfiles(...)	Activate additional Spring profiles.
setBannerMode(...)	Control how the startup banner is displayed.
run(...)	Launches the application.

KEY TAKEAWAY SpringApplication is the bootstrapper — it prepares the environment, builds the context, and starts the server.

EMBEDDED SERVER ARCHITECTURE — HOW IT WORKS

Spring Boot comes with an embedded servlet container so you can build and run applications without deploying a WAR file.



KEY CHARACTERISTICS

- Starts automatically when the application starts.
- Listens on a configurable port (default: 8080).
- Can be customized via application.properties/yml.
- Gracefully shuts down with the application.

KEY TAKEAWAY The embedded server is built into your Spring Boot app — it starts, runs, and stops with your application.

Spring Boot Embedded Server Architecture

Spring Boot comes with an embedded server, so you can build and run web applications without deploying an external application server.

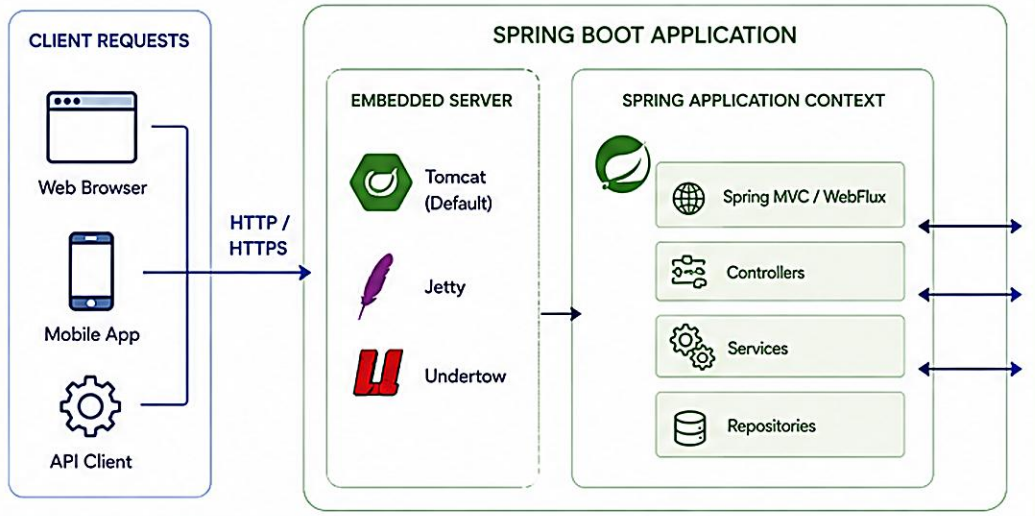
WHAT IS AN EMBEDDED SERVER?

An embedded server is built-in to your Spring Boot application. It starts with your application and stops when your application stops.

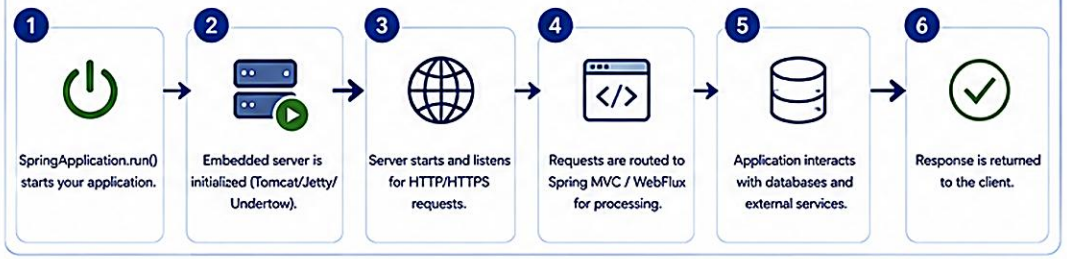
KEY BENEFITS

- Zero external server installation
- Easy to build, run and deploy
- Self-contained and portable
- Perfect for microservices and cloud-native apps
- Consistent across environments

EMBEDDED SERVER ARCHITECTURE



HOW IT WORKS



CONFIGURE EMBEDDED SERVER

Change Port

```
server.port=8081
```

Context Path

```
server.servlet.context-path=/app
```

Choose Embedded Server

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-web</artifactId>
</dependency>
```

Exclude Default (Tomcat) and Add Jetty

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-web</artifactId>
  <exclusions>
    <exclusion>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-starter-tomcat</artifactId>
    </exclusion>
  </exclusions>
</dependency>

<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-jetty</artifactId>
</dependency>
```

BOTTOM LINE

Embedded servers in Spring Boot make your application self-contained, easy to run, and production-ready.

DEFAULT EMBEDDED SERVER

Apache Tomcat (Included by default)

SUPPORTED EMBEDDED SERVERS

Jetty, Undertow, Netty

TIP

Great for microservices, serverless, and containerized applications!

EMBEDDED SERVER ARCHITECTURE — SERVERS & CONFIGURATION

Only one embedded server should be on the classpath at a time — Tomcat is chosen by default if multiple are present.

Server	Starter Dependency	Notes
Tomcat (Default)	spring-boot-starter-web (includes tomcat-embed-core)	Most widely used, robust and feature-rich.
Jetty	spring-boot-starter-jetty	Lightweight and fast — good for high concurrency.
Undertow	spring-boot-starter-undertow	Non-blocking I/O — high performance.

Configuration Example (application.properties)

```
server.port=8080
server.servlet.context-path=/app
server.tomcat.max-threads=200
server.tomcat.accept-count=100
server.compression.enabled=true
```

BENEFITS

Self-contained | Easy to Deploy | Consistent

Use external servers only for specific enterprise needs (clustering, advanced security). For most microservices, embedded servers are the best choice.

KEY TAKEAWAY Spring Boot includes a built-in server (Tomcat by default) that starts with your app and handles all incoming HTTP requests.

TRY IT YOURSELF — EXERCISE 4: APPLICATION.YML SKETCH

Reinforces: the server.port and configuration keys you just covered — no secrets committed.

STEP	WHAT TO DO
Goal	Sketch the YAML keys Lab 23 will use, without committing secrets.
File	notes/application-yml-sketch.yml (in examples/module-23-exercises/)
Keys to draft	spring.application.name = northstar-crm; server.port = 8080; a logging level.
Profile teaser	Add a commented line mentioning spring.profiles.active — Lab 26 deepens this; no invented prod secrets.
Security hygiene	Write it down: real DB passwords never go in committed YAML.
Reference	exercises/exercise-04-application-yml-sketch.md

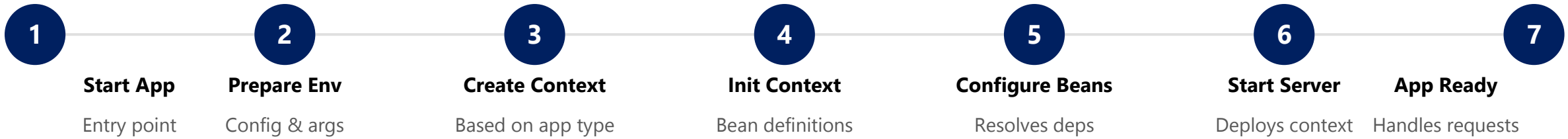
notes/application-yml-sketch.yml

```
spring:
  application:
    name: northstar-crm
server:
  port: 8080
logging.level.root: INFO
# spring.profiles.active: dev   <- teaser only; Lab 26 owns real profiles/secrets
```

TRY IT NOW Complete Exercise 4 before continuing — see exercises/exercise-04-application-yml-sketch.md.

APPLICATION STARTUP SEQUENCE — THE 7-STEP FLOW

When you run a Spring Boot application, a series of well-defined steps bootstrap the environment and start serving requests.



WHY IT MATTERS

- Understanding the sequence helps in debugging startup issues.
- Enables advanced customization and improves performance tuning.

KEY TAKEAWAY Spring Boot follows a structured startup sequence to create a fully initialized application ready to handle requests.

Spring Boot Application Startup Sequence

Spring Boot follows a well-defined startup process to initialize and start your application quickly and reliably.

APPLICATION STARTUP FLOW

1. **Start Application** — The main method calls SpringApplication.run()
2. **Create SpringApplication** — SpringApplication object is created and initialized with the application sources.
3. **Configure Environment** — Environment is prepared by loading properties from application.properties, profiles, system env, etc.
4. **Load Application Context** — The ApplicationContext is created and prepared (AnnotationConfigServletWebServerApplicationContext by default).
5. **Process Bean Definitions** — Component scanning finds @Component, @Service, @Repository, @Controller, etc. Bean definitions are registered.
6. **Instantiate Beans** — Beans are instantiated, dependencies are injected and configured. Post processors are applied.
7. **Start Embedded Server** — Tomcat/Jetty/Undertow is started on the configured port.
8. **Run Initializers** — Application runners and command line runners are executed.
9. **Application Ready** — Application is ready to serve requests.
10. **Listen for Requests** — The application is now running and listening for HTTP requests.

KEY EVENTS PUBLISHED

Event	Purpose
ApplicationStartingEvent	Published at the very beginning of startup.
ApplicationEnvironmentPreparedEvent	After environment is prepared but before context is created.
ApplicationContextInitializedEvent	After ApplicationContext is initialized.
ApplicationPreparedEvent	Before the context is refreshed.
ApplicationStartedEvent	After the context is refreshed and started.
ApplicationReadyEvent	Application is ready to service requests.
ApplicationFailedEvent	Published if the startup fails at any point.

DETAILED STARTUP TIMELINE

1. SpringApplication.run(DemoApplication.class)
2. Create SpringApplication instance
3. Set active profiles and configure Environment
4. Create and configure ApplicationContext
5. Scan components and register bean definitions
6. Create beans and resolve dependencies
7. Start embedded web server
8. Execute ApplicationRunner and CommandLineRunner
9. Publish ApplicationReadyEvent
10. Application is live and ready to handle requests

WHAT HAPPENS INSIDE?

- **Environment** — Load config, profiles, properties
- **ApplicationContext** — BeanFactory, Resource loading, Post Processors
- **Bean Lifecycle** — Instantiate, Configure, Initialize, Inject
- **Embedded Server** — Start web server and deploy context
- **Application Ready** — Accept incoming requests

CUSTOMIZATION POINTS

- **ApplicationContextInitializer** — Customize the context before it is refreshed.
- **EnvironmentPostProcessor** — Modify environment properties.
- **BeanFactoryPostProcessor** — Change bean definitions before instantiation.
- **ApplicationRunner / CommandLineRunner** — Run custom logic after the context is ready.
- **@Order / @DependsOn** — Control order and dependencies.
- **Profiles** — Load profile-specific configurations.

BOTTOM LINE

Spring Boot automates the complex startup process, so you can focus on building great applications.

OUTCOME

A fully initialized application, running on an embedded server, ready to serve requests with minimal configuration.

TIP

Enable debug logs (`logging.level.org.springframework=DEBUG`) to see detailed startup logs and diagnose issues.

APPLICATION STARTUP SEQUENCE — EVENTS PUBLISHED

Spring Boot publishes lifecycle events at each stage of startup — useful hooks for logging, metrics, and custom initialization.

Event	Description
ApplicationStartingEvent	Published at the very beginning of the run.
ApplicationEnvironmentPreparedEvent	Published after the Environment is prepared.
ApplicationContextInitializedEvent	Published when the ApplicationContext is initialized.
ApplicationPreparedEvent	Published after the context is prepared, before refresh.
ApplicationStartedEvent	Published after the context is refreshed and the server has started.
ApplicationReadyEvent	Published when the application is ready to service requests.
ApplicationFailedEvent	Published if startup fails.

BEST PRACTICE

- Keep startup fast — avoid heavy logic in @PostConstruct.
- Use Lazy Initialization for large applications and monitor startup time in production.

KEY TAKEAWAY Enable debug logging to see detailed startup logs and the time taken by each step.

SPRING BOOT STARTER PARENT

A special parent POM provided by Spring Boot that gives every project sensible defaults and best practices.

WHAT YOU GET OUT OF THE BOX

- **Managed Dependency Versions —** no need to specify versions for Spring, Jackson, Tomcat, JUnit, and more.
- **Default Plugin Configurations —** compiler, surefire, jar, and other essential Maven plugins.
- **Java & Encoding Defaults —** default Java version (17+) and UTF-8 encoding.
- **Resource Handling —** filtering and static resource defaults.

pom.xml

```
<parent>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-parent</artifactId>
  <version>3.3.0</version>
  <relativePath/>
</parent>

<properties>
  <java.version>17</java.version>
</properties>

<!-- You typically do NOT declare versions
     for Spring Boot starter dependencies -->
```

KEY TAKEAWAY The starter parent is a blueprint — it manages versions and build details so you can focus on your application.

SPRING BOOT STARTER WEB

One of the most commonly used starters — provides everything needed to build web applications and RESTful services with Spring MVC.

pom.xml

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-web</artifactId>
</dependency>
```

WHAT YOU GET AUTOMATICALLY

- Spring MVC for REST APIs and web apps.
- Embedded Tomcat server (default).
- Jackson for JSON serialization.
- Validation and built-in error handling.

Common properties: server.port=8080 | server.servlet.context-path=/api — customize the embedded server per environment.

KEY TAKEAWAY Add one dependency and get a fully functional web stack with embedded Tomcat and Spring MVC — no extra configuration required.

SPRING BOOT STARTER TEST

Provides essential testing libraries and frameworks so you can write and run unit, integration, and web layer tests with ease.

pom.xml

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-test</artifactId>
  <scope>test</scope>
</dependency>
```

Library	Purpose
JUnit 5	Modern testing framework with annotations.
AssertJ	Fluent assertions for readable tests.
Mockito	Mocking framework for isolating dependencies.
Spring Test	TestContext, DI, and profiles in tests.

TYPICAL TESTS YOU CAN WRITE

Unit Tests | **Data Layer (@DataJpaTest)** | **Web Layer (@WebMvcTest)** | **Integration (@SpringBootTest)**

BEST PRACTICES

- Prefer constructor injection and immutable dependencies to simplify testing.
- Keep tests fast, isolated, and deterministic with descriptive names.

KEY TAKEAWAY spring-boot-starter-test gives you all the testing superpowers you need — JUnit, Mockito, AssertJ — in one dependency. Scope: test.

TRY IT YOURSELF — EXERCISE 5: REST SMOKE PLAN

Reinforces: testing a running app's first endpoint — the integration-test types you just covered.

STEP	WHAT TO DO
Goal	Document the Lab 23 HTTP smoke sequence without executing the full lab.
File	notes/rest-smoke-plan.md (in examples/module-23-exercises/)
Sequence	Start app -> POST Amina -> GET CUS-1001 -> GET CUS-1002 (or create Ravi) -> GET missing -> check health.
Correlation	Specify header X-Correlation-Id: lab-request-001 on the create evidence.
Failure case	Note the expected 404 for CUS-MISSING (or equivalent missing id).
Reference	exercises/exercise-05-rest-smoke-plan.md

notes/rest-smoke-plan.md (sequence)

```
1. Start app  -> mvn spring-boot:run
2. POST /api/customers (Amina; X-Correlation-Id: lab-request-001)
3. GET  /api/customers/CUS-1001   // Amina Khan, ACTIVE
4. GET  /api/customers/CUS-1002   // Ravi Singh, PROSPECT (or create)
5. GET  /api/customers/CUS-MISSING -> expect 404
6. GET  /actuator/health           -> expect UP
// SOAP partner calls wait for Lab 24
```

TRY IT NOW Complete Exercise 5 before continuing — see exercises/exercise-05-rest-smoke-plan.md.

SPRING BOOT STARTER VALIDATION

Adds Bean Validation (JSR 380) support to your application with Hibernate Validator as the default implementation.

pom.xml

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-validation</artifactId>
</dependency>
```

Annotation	Description
@NotNull / @NotBlank	Must not be null / not empty (trimmed).
@Size(min, max)	Size must be between min and max.
@Email	Must be a valid email.
@Positive	Value must be greater than 0.

DTO with Validation + Controller with @Valid

```
public class UserRequest {
    @NotBlank(message = "Name is required") private String name;
    @Email(message = "Invalid email address") private String email;
}

@PostMapping("/users")
public ResponseEntity<String> createUser(@Valid @RequestBody UserRequest request) { ... }
// If validation fails, Spring returns HTTP 400 Bad Request automatically
```

KEY TAKEAWAY spring-boot-starter-validation adds powerful validation with almost no code — annotate your fields and Spring handles the rest.

INTRODUCTION TO SPRING BOOT ACTUATOR

Actuator exposes production-ready endpoints to help you monitor and manage your application at runtime.

Spring Boot Actuator adds built-in endpoints for health, metrics, and diagnostics with almost no extra code.

How to Enable — pom.xml

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-actuator</artifactId>
</dependency>
```

Expose Endpoints — application.properties

```
management.endpoints.web.exposure.include=health,info
management.endpoint.health.show-details=when_authorized
```

KEY ACTUATOR ENDPOINTS

Endpoint	Purpose
/actuator/health	Application health status (UP / DOWN).
/actuator/info	Application information (custom metadata).
/actuator/metrics	Application and JVM performance metrics.
/actuator/env	Environment properties and configuration sources.

KEY TAKEAWAY Actuator turns a black-box JAR into an observable, production-ready service — expose only what you need, and secure it.

Spring Boot Actuator

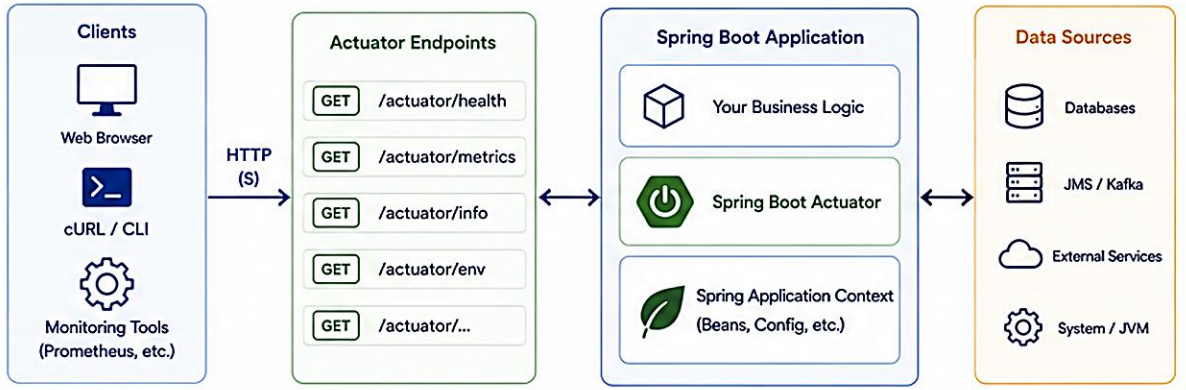
Production-ready features to help you monitor and manage your Spring Boot application.

WHAT IS ACTUATOR?

Spring Boot Actuator provides built-in endpoints to monitor, manage, and interact with your application in production.

It exposes operational information about your application via HTTP or JMX.

HOW ACTUATOR WORKS



KEY FEATURES

Health Monitoring
Check the health of your application and its dependencies.

Metrics Collection
Expose metrics for performance monitoring and alerting.

Application Information
View build info, environment properties, and more.

Production Management
Control logging levels, thread dumps, heap dumps, and more.

Secure by Default
Endpoints are not exposed by default (except /health and /info).

Highly Extensible
Add custom endpoints, health checks, metrics, and more.

COMMON ENDPOINTS

Endpoint	Description	Example Output
/actuator/health	Shows the health status of the application.	`{ "status": "UP", "components": { "db": { "status": "UP" } }`
/actuator/info	Displays application information.	`{ "app": { "name": "DemoApp", "version": "1.0.0" } }`
/actuator/metrics	Lists available metrics and their values.	`{ "names": [ "jvm.memory.used", "http.server.requests" ] }`
/actuator/env	Shows environment properties.	`{ "propertySources": [ ... ], "activeProfiles": [ "prod" ] }`
/actuator/beans	Displays all Spring beans in the context.	`{ "contexts": { "application": { "beans": { "myService": { ... } } } }}`
/actuator/threaddump	Returns the thread dump of the JVM.	`Full thread dump text`
/actuator/loggers	View and modify logging levels.	`{ "configuredLevel": "INFO", "effectiveLevel": "INFO" }`

SECURING ACTUATOR ENDPOINTS

Actuator endpoints should be secured in production.

- Use Spring Security
- Expose only necessary endpoints
- Restrict access by role or IP
- Run Actuator on a different port (optional)

application.yml example

```yaml
management:
  endpoints:
    web:
      exposure:
        include: health,info,metrics,prometheus
  endpoint:
    health:
      show-details: when_authorized
server:
  port: 8080
management:
  server:
    port: 8081    # Actuator on a different port
```

OBSERVABILITY INTEGRATION

Prometheus
Scrape metrics from /actuator/prometheus for monitoring and alerting.

Grafana
Visualize metrics collected via Actuator endpoints.

ELK Stack
Send logs and metrics to Elasticsearch, Logstash, and Kibana.

Alerts
Set up alerts based on Actuator metrics (e.g., high CPU, errors, slow requests).

BOTTOM LINE

Spring Boot Actuator gives you deep visibility into your application's health, performance, and operations with minimal effort.

DEFAULT EXPOSED ENDPOINTS

By default, only /health and /info are exposed. All others must be explicitly enabled.

CUSTOM ENDPOINT EXAMPLE

```java
@Endpoint(id = "hello")
public class HelloEndpoint {
  @ReadOperation
  public String hello() { return "Hello Actuator!"; }
}
```

TIP

Actuator is your operational dashboard. Use it, secure it, and monitor your application like a pro!

HEALTH ENDPOINTS

Spring Boot Actuator — health endpoints provide real-time information about the health status of your application and its components.

URL	Description
GET /actuator/health	Basic health status (UP / DOWN).
GET /actuator/health/{component}	Health of a specific component.
GET /actuator/health?details=true	Detailed health information.

COMMON STATUS VALUES

UP | DOWN | OUT_OF_SERVICE | UNKNOWN

GET /actuator/health?details=true

```
{ "status": "UP", "components": {
  "db": { "status": "UP", "details": { "database": "MySQL" } },
  "diskSpace": { "status": "UP", "details": { "free": 53687091200 } }
} }
```

DEFAULT HEALTH INDICATORS

db | diskSpace | ping | process | livenessState | readinessState

KEY TAKEAWAY The /actuator/health endpoint gives a clear, structured view of your application's health and its dependencies.

Spring Boot Actuator – Health Endpoints

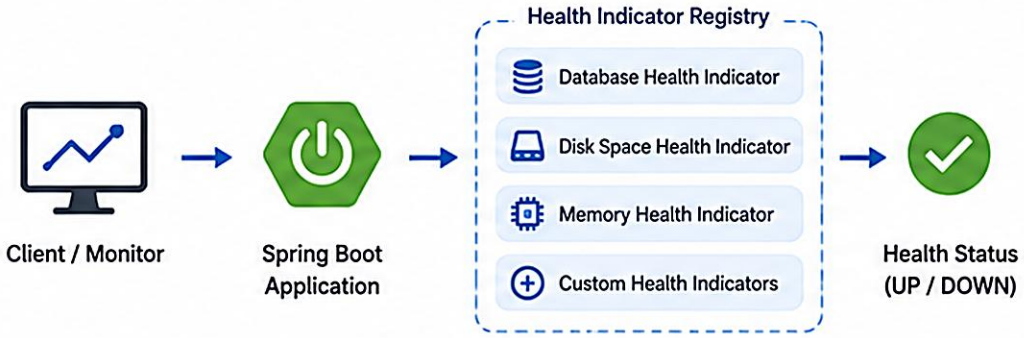
Health endpoints provide real-time status of application and its components

OVERVIEW

Health endpoints expose the health status of your application and its dependencies.

They are essential for monitoring, alerting and automated recovery in production environments.

HEALTH ENDPOINT FLOW



COMMON HEALTH ENDPOINTS

Endpoint	Description	Access
/actuator/health	Overall health status	GET
/actuator/health/db	Database health	GET
/actuator/health/diskspace	Disk space health	GET
/actuator/health/memory	Memory health	GET
/actuator/health/liveness	Liveness probe	GET
/actuator/health/readiness	Readiness probe	GET

EXAMPLE RESPONSE – /actuator/health

```
{
  "status": "UP",
  "components": {
    "db": { "status": "UP", "details": { "database": "PostgreSQL" } },
    "diskSpace": { "tatus": "UP", "details": { "free": "10GB" } },
    "ping": { "status": "UP" },
    "memory": { "status": "UP", "details": { "used": "256MB", "max": "1024MB" } }
  }
}
```

STATUS VALUES

UP
The component is functioning correctly.

OUT_OF_SERVICE
The component is up but having issues.

DOWN
The component is not functioning.

USE CASES

- Monitoring application and dependency health in real-time
- Integrating with alerting systems (Prometheus, Grafana, etc.)
- Kubernates liveness and readiness probes
- Automated recovery and self-healing strategies

KEY POINTS

- Actuator health endpoints expose application and dependency status.
- Easily integrate with monitoring and alerting tools.
- Essential for production readiness and reliability.
- Custom health indicators can be added.

METRICS ENDPOINTS

Spring Boot Actuator — metrics endpoints expose real-time metrics about performance, resource utilization, and system behavior.

Endpoint	Description
GET /actuator/metrics	List all available metrics.
GET /actuator/metrics/{name}	Get a specific metric, e.g. jvm.memory.used.
GET /actuator/metrics/{name}?tag=...	Get metric filtered by tag.
GET /actuator/prometheus	Metrics in Prometheus scrape format.

COMMON METRICS

System | JVM | HTTP | DataSource | Disk Space | Custom

GET /actuator/metrics/jvm.memory.used

```
{ "name": "jvm.memory.used", "baseUnit": "bytes",
  "measurements": [ { "statistic": "VALUE", "value": 1.23456789E8 } ],
  "availableTags": [ { "tag": "area", "values": ["heap", "nonheap"] } ] }
```

Micrometer exposes metrics in Prometheus format for seamless integration with Prometheus, Grafana, Datadog, CloudWatch, and New Relic.

KEY TAKEAWAY The /actuator/metrics endpoint and Micrometer provide a rich set of application and JVM metrics for monitoring tools.

Spring Boot Actuator – Metrics Endpoints

Metrics endpoints expose application and JVM metrics for monitoring and observability

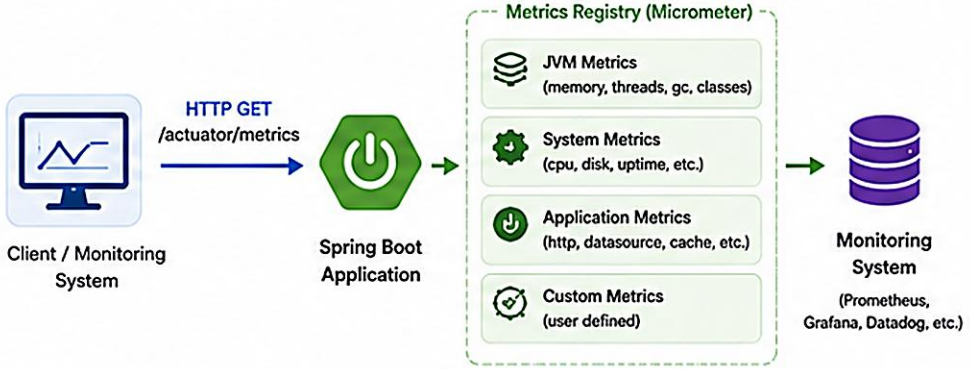
Built-in Metrics | Custom Metrics | Export to Monitoring Systems

OVERVIEW

Metrics endpoints provide detailed quantitative data about your application, JVM, and system performance.

- Track performance and resource utilization in real time
- Identify bottlenecks and capacity issues
- Integrate with monitoring systems like Prometheus, Grafana, Datadog, etc.

METRICS ENDPOINT FLOW



COMMON METRICS ENDPOINTS

Endpoint	Description	Example Metrics
/actuator/metrics	List all available metrics	—
/actuator/metrics/{name}	Details of a specific metric	http.server.requests
/actuator/metrics/jvm.memory.used	JVM memory used	used, committed, max
/actuator/metrics/jvm.gc.pause	JVM GC pause time	count, totalTime
/actuator/metrics/system.cpu.usage	CPU usage	value
/actuator/metrics/http.server.requests	HTTP request metrics	count, max, mean
/actuator/metrics/hikaricp.connections	DB connection pool metrics	active, idle, max
/actuator/prometheus	Prometheus format output	All metrics

EXAMPLE RESPONSE – /actuator/metrics/http.server.requests

```
{
  "name": "http.server.requests",
  "description": "Counts the number of HTTP server requests",
  "baseUnit": "requests",
  "measurements": [
    { "statistic": "COUNT", "value": 12543 },
    { "statistic": "TOTAL_TIME", "value": 34567.89" },
    { "statistic": "MAX", "value": 1234.0 }
  ],
  "availableTags": [
    { "tag": "method", "values": ["GET", "POST", "PUT", "DELETE"] },
    { "tag": "status", "values": ["200", "404", "500"] },
    { "tag": "uri", "values": ["/api/**", "/actuator/**"] }
  ]
}
```

TYPES OF METRICS

GAUGE
A single numerical value that can go up or down.
Example: system.cpu.usage

COUNTER
A monotonically increasing value.
Example: http.server.requests

TIMER
Measures duration and provides statistics.
Example: http.server.requests

DISTRIBUTION SUMMARY
Tracks distribution of values.
Example: jvm.gc.pause

USE CASES

- Real-time performance monitoring and alerting
- Capacity planning and trend analysis
- Detect bottlenecks and performance issues
- SLA monitoring and reliability tracking
- Business and operational observability

EXPORT METRICS

Spring Boot Actuator metrics can be exported in multiple formats.

- **Prometheus** /actuator/prometheus
- **Micrometer Registry** (Prometheus, Datadog, New Relic, etc.)
- **Datadog** via Micrometer Datadog Registry
- **Graphite / Influx** via Micrometer Registry
- **Logs / Custom** via Logging or custom export

KEY POINTS

- Expose rich operational metrics with minimal configuration.
- Built on Micrometer – vendor neutral and extensible.
- Monitor JVM, system, application and custom business metrics.
- Integrate seamlessly with Prometheus, Grafana and more.
- Enable data-driven performance optimization and reliability.

SPRING BOOT BEST PRACTICES (1 OF 2)

Following best practices helps you build applications that are secure, maintainable, scalable, and easy to operate.

#	Practice	What It Means
1	Follow a Clean Project Structure	Use a standard layered architecture; keep packages small and focused.
2	Externalize Configuration	Use application.properties/yml and profiles for dev/test/prod; never hardcode values.
3	Write Clean, Maintainable Code	Follow SOLID principles; use meaningful names and small, focused methods.
4	Use Dependency Injection Properly	Prefer constructor injection; depend on abstractions, not concrete classes.
5	Test Your Code	Write unit and integration tests; maintain good coverage for critical logic.

KEY TAKEAWAY Consistent structure, externalized config, and clean code are the foundation of maintainable Spring Boot applications.

SPRING BOOT BEST PRACTICES (2 OF 2)

Apply these practices consistently to deliver high-quality, production-ready applications with minimal surprises.

#	Practice	What It Means
6	Secure Your Application	Use Spring Security; externalize secrets; validate and sanitize all inputs.
7	Monitor and Observe	Enable Actuator endpoints; integrate with Prometheus, Grafana, or other tools.
8	Handle Exceptions Gracefully	Use @ControllerAdvice for global handling; return meaningful error responses.
9	Optimize Performance	Use caching, DTOs to reduce payload size, and avoid N+1 queries.
10	Prepare for Production	Use a production-ready database, connection pool, timeouts, and secrets management.

KEY TAKEAWAY Security, observability, and performance discipline turn a working app into a production-ready one.

PRODUCTION READINESS CONSIDERATIONS

Production environments demand high availability, security, performance, and observability — plan for them early.

Externalize Config

Use YAML with placeholders and environment variables — never hardcode secrets.

Secure Application

Spring Security, encryption in transit/at rest, input validation.

Observability

Actuator, Prometheus/Grafana, centralized logs, alerts.

Optimize Performance

Caching, connection pools (HikariCP), avoid N+1 queries.

Database & Data Mgmt

Production-grade RDBMS, backups, Flyway/Liquibase migrations.

Resilience

Timeouts, retries, Resilience4j, graceful degradation.

Logging Best Practices

Structured (JSON) logs with correlation IDs; avoid logging secrets.

Health & Readiness

Custom indicators; liveness/readiness probes for Kubernetes.

Deployment & Release

CI/CD pipelines, blue-green/canary deploys, rollback plans.

Resource & Scalability

JVM tuning, autoscaling, stateless services for horizontal scale.

KEY TAKEAWAY A production-ready Spring Boot app is secure, observable, configurable, performant, and easy to operate and evolve.

PRODUCTION READINESS — ACTUATOR ENDPOINTS & CHECKLIST

Essential Actuator endpoints for production, and the checklist to confirm before go-live.

Endpoint	Purpose	Access
/actuator/health	Application health status	Public (limited)
/actuator/info	Application information	Public
/actuator/metrics	Application metrics	Restricted
/actuator/prometheus	Prometheus metrics	Restricted
/actuator/env	Environment properties	Restricted

PRODUCTION CHECKLIST

- Externalized configuration & secrets management.
- Security (auth, authz, encryption, validation).
- Database backups & migration strategy.
- Health checks, metrics, logging enabled.
- Performance tuning & load testing done.
- Resilience patterns implemented.
- CI/CD pipeline & rollback plan ready.
- Monitoring, alerts, dashboards configured.

Restrict sensitive endpoints (env, metrics, prometheus) with Spring Security in production.

KEY TAKEAWAY Production readiness is a mindset — secure everything by default, monitor everything that matters, test for failure.

TRY IT YOURSELF — EXERCISE 6: LAB 23 READINESS CHECKLIST (CAPSTONE)

Capstone check — confirm tools and the lab23-crm path before you start the graded lab.

STEP	WHAT TO DO
Goal	Ready the workspace for the Initializr-style Lab 23 without completing it.
File	notes/lab23-readiness.md (in examples/module-23-exercises/)
Versions	Record JDK 21 and Maven 3.9+.
Copy path	Write the target: examples/lab23-crm under java-bootcamp.
Depends on / Not yet	Lab 22 constructor-DI habits should already be understood; defer SOAP WSDL, JWT login, @Transactional.
Reference	exercises/exercise-06-lab23-readiness.md

notes/lab23-readiness.md

```
$ java -version   // expect JDK 21
$ mvn -version    // expect Maven 3.9+
Target: examples/lab23-crm  (under java-bootcamp)
Depends on: Lab 22 constructor-DI habits
Not yet: SOAP WSDL, JWT login, @Transactional -- later labs
```

TRY IT NOW Complete Exercise 6 before continuing to Lab 23 — see exercises/exercise-06-lab23-readiness.md.

LAB OVERVIEW — SPRING BOOT SETUP AND AUTO-CONFIGURATION

Lab 23: Spring Boot Setup and Auto-Configuration — Northstar CRM First Boot App

BUSINESS SCENARIO

- Northstar needs a single runnable Boot process: starters, application.yml, an embedded server, REST /api/customers, and Actuator health.
- You own the slice for Amina Khan (CUS-1001, ACTIVE) and Ravi Singh (CUS-1002, PROSPECT), plus a 404 path for missing IDs.
- Peers must start it with mvn spring-boot:run, hit create/get, and smoke-check with /actuator/health.

LEARNING OBJECTIVES

- Create a Spring Boot 3.x project (Initializr-style) with web, actuator, and test starters.
- Configure application.yml for server port, application name, and basic logging.
- Implement a first REST API for customers using Boot auto-configured MVC.
- Verify Actuator health as a smoke check; introduce dev/prod profiles as a teaser.
- Explain what auto-configuration did for you and what you still own.

WHAT YOU BUILD

- Scaffold lab23-crm; pin web + actuator + test (+validation); write CrmApplication and application.yml; implement in-memory customer create/get with correlation lab-request-001; verify health; add dev/prod profile teasers; automate context-load + HTTP IT.

KEY TAKEAWAY Fixtures CUS-1001 (Amina Khan) and CUS-1002 (Ravi Singh) with correlation lab-request-001 anchor every example.

LAB TASKS AND DELIVERABLES

Northstar CRM First Boot App — implementation steps, deliverables, and the evaluation rubric.

#	Implementation Step
1	Create the Initializr-style project — Boot parent, Java 21, web/actuator/test starters.
2	Add CrmApplication and prove Boot starts (embedded Tomcat on 8080).
3	Configure application.yml — app name, port, Actuator exposure, logging.
4	Implement Customer model and in-memory CustomerService (constructor DI).
5	Expose /api/customers create and get; echo X-Correlation-Id; 201/200/404.
6	Verify Actuator health and info endpoints.
7	Add dev/prod profile teasers (log level, health detail, Actuator exposure).
8	Automate Boot smoke tests (context-load + HTTP IT) and failure experiments.

EXPECTED DELIVERABLES

- Initializr-style pom.xml with web + actuator + test.
- CrmApplication + application.yml + profile teasers.
- /api/customers evidence for CUS-1001/CUS-1002/lab-request-001.
- Actuator health verification; automated context-load + API IT.
- Autoconfig-vs-ownership notes; controlled-failure evidence.

RUBRIC (100 MARKS)

- Environment 10 · Core Impl 30 · Integration/Config 15 · Failure Handling 15 · Verification 10 · Security 10 · Docs 10

KEY TAKEAWAY A green package without HTTP evidence for CUS-1001/CUS-1002 loses core marks — committing secrets is an honor violation.

MODULE 23 SUMMARY

Spring Boot Auto-Configuration — what we covered and how it fits together.

KEY CONCEPTS COVERED

Spring Boot Fundamentals

What Spring Boot is, Spring vs Spring Boot, benefits and architecture.

Auto-Configuration

Classpath scanning, conditional annotations, and starter dependencies.

Lifecycle & Embedded Server

SpringApplication, embedded Tomcat, and the application startup sequence.

Starter Dependencies

Parent, Web, Test, and Validation starters and what each provides.

Actuator (Production Ready)

Health, metrics, and other endpoints for runtime inspection.

Best Practices

Security, observability, performance, and production readiness.

MODULE FLOW RECAP

1. **Starters**
2. **Auto-Configuration**
3. **Embedded Server**
4. **Bootstrap & Lifecycle**
5. **Actuator & Production Readiness**

KEY TAKEAWAY Spring Boot's auto-configuration does the heavy lifting so you can focus on business logic — production-ready with minimal effort.

KNOWLEDGE CHECK (1 OF 2)

Test your understanding of Spring Boot auto-configuration and starter dependencies.

1. What is the primary goal of Spring Boot auto-configuration?

A. To write configuration files for the developer

B. Auto-configure components based on the classpath

C. To replace all XML configuration

D. To prevent the use of annotations

2. Which annotation applies configuration only when a condition is met?

A. @Autowired

B. @Conditional

C. @Component

D. @Bean

3. Which starter provides Spring MVC and an embedded Tomcat server?

A. spring-boot-starter-data-jpa

B. spring-boot-starter-web

C. spring-boot-starter-security

D. spring-boot-starter-actuator

4. Which starter performs validation using Jakarta Bean Validation (Hibernate Validator)?

A. spring-boot-starter-validation

B. spring-boot-starter-test

C. spring-boot-starter-web

D. spring-boot-starter-data-jpa

KEY TAKEAWAY Auto-configuration and starters are the foundation of every Spring Boot application.

KNOWLEDGE CHECK (2 OF 2)

Four more questions on classpath behavior, testing, embedded servers, and Actuator.

5. True or False: Spring Boot auto-configuration is applied regardless of what is on the classpath.

A. True

B. False — conditions must match the classpath

6. True or False: spring-boot-starter-test is intended for production use.

A. True

B. False — has "test" scope, excluded from prod

7. You added spring-boot-starter-web and started the app with no server config. Why did it start successfully?

A. Spring Boot requires you to manually install Tomcat first

B. Auto-config finds the web starter, adds Tomcat

C. Spring Boot runs with no server by default

D. The IDE silently installs a server

8. Which Actuator endpoint returns the health status of the application?

A. /actuator/metrics

B. /actuator/beans

C. /actuator/health

D. /actuator/env

KEY TAKEAWAY Actuator and conditional configuration make your Spring Boot app observable and production-ready.

"Spring Boot takes care of the infrastructure, so you can focus on what matters — your business logic."

MODULE 23 COMPLETE

Spring Boot Auto-Configuration

You can now create, configure, run, and monitor production-ready Spring Boot applications using starters, auto-configuration, embedded servers, and Actuator.